

PayMyLoan — Plan de Implementación de Frontend

Estado: Plan de trabajo para revisión. No implementado. No se modificó ningún archivo del proyecto como parte de este documento. **Fuentes analizadas:** `paymyloan-alcance.html` v0.1, código real del proyecto actual `paymyloan` (Next.js 16, landing de registro), y `paymyloan-back/PAYMYLOAN_BACKEND_IMPLEMENTATION_PLAN.md` (documento hermano de este — toda la integración con API descrita aquí asume exactamente los endpoints, roles y modelo de datos definidos ahí). **Compañero de este documento:** `paymyloan-back/PAYMYLOAN_BACKEND_IMPLEMENTATION_PLAN.md`.

0. Decisiones de las que depende este plan

Este plan no repite el análisis de contradicciones (ya resuelto en la sección 0 del plan de backend) — hereda directamente esas tres decisiones:

- 1. **Roles jerárquicos:** Admin administra Prestamistas (tenants); el Prestamista crea el Contrato y lo somete a aceptación; el Deudor **debe aceptar o rechazar formalmente** los términos antes de que el contrato quede activo.
- 2. **UUIDv7:** sin impacto directo en el frontend salvo que todo ID en la UI (URLs, params) es un string UUID, no un cuid — no cambia ningún componente, solo la forma del dato.
- 3. **Aceptación bilateral obligatoria:** el frontend del Deudor necesita una pantalla dedicada, bloqueante, para revisar y aceptar/rechazar términos — no es un detalle menor, es una de las vistas más importantes del rol Deudor (sección 5.3).

Además, este plan toma tres decisiones de arquitectura propias, no pedidas explícitamente pero necesarias para poder diseñar el resto — se marcan como confirmables en la sección 12, no bloquean la redacción del plan:

#	Decisión	Motivo
F-1	La app autenticada (Admin/Prestamista/Deudor) vive dentro del mismo proyecto <code>paymyloan</code> que la landing pública, como rutas nuevas (<code>/login</code> , <code>/admin/*</code> , <code>/lender/*</code> , <code>/borrower/*</code>), reutilizando el Docker/nginx/certbot ya existente	El encargo habla de "el proyecto actual de PayMyLoan" en singular; <code>paymyloan-back</code> sí es un proyecto separado (ya lo es hoy), pero nada indica que la landing y la app deban ser dos repos de frontend distintos, y partirlo duplicaría build/despliegue sin necesidad clara
F-2	Sesión con refresh token en cookie httpOnly + access token en memoria (no <code>localStorage</code>), gestionada por un puñado de Route Handlers propios del frontend (<code>/api/session/*</code>) que actúan como intermediarios solo para login/refresh/logout — el resto de las llamadas van directas del navegador a <code>paymyloan-back</code>	El alcance marca la seguridad como prioridad transversal y el dominio maneja datos financieros; un token en <code>localStorage</code> es robable por XSS, un refresh token en cookie httpOnly no. Construir un proxy para los ~50 endpoints de negocio sería sobre-ingeniería — solo auth lo necesita

#	Decisión	Motivo
F-3	La app autenticada se construye solo en español para el MVP, con todo el texto centralizado (no bilingüe todavía)	La pregunta de bilingüismo (alcance Q13) sigue abierta y el encargo de este plan está redactado enteramente en español, sin mencionar inglés para las vistas de rol. Se mantiene el copy centralizado (mismo patrón que ya usa la landing en <code>content/copy.ts</code>) para que traducir después sea agregar un diccionario, no reescribir componentes

1. Arquitectura frontend

1.1 Punto de partida real (verificado en el código actual)

`paymyloan` hoy es **únicamente** una landing pública de registro: Next.js 16 (App Router, Turbopack), React 19, Tailwind v4, `lucide-react`, sin Prisma/NextAuth/Stripe, con:

- Sistema de diseño propio ya definido en `app/globals.css` (tokens de color claro/oscura, tipografía Spectral + IBM Plex Sans + IBM Plex Mono) — **se reutiliza tal cual**, no se reinventa un sistema de diseño nuevo para la app autenticada.
- Copy bilingüe centralizado en `content/copy.ts` — se sigue el mismo patrón para el copy nuevo (aunque monolingüe por ahora, ver F-3).
- Un único punto de salida hacia un backend (`app/lib/registro.ts`), hoy simulado porque `NEXT_PUBLIC_REGISTRO_ENDPOINT` está vacío — se reemplaza por el cliente HTTP real de la sección 7, y el propio formulario de registro de la landing puede quedar obsoleto o redirigir a un flujo de alta real según decida el negocio (fuera de alcance de este plan, ver sección 12).
- Docker + nginx + certbot ya funcionando para `paymyloan.ai` — se reutiliza sin cambios de infraestructura (la app autenticada son rutas nuevas dentro del mismo build).

1.2 Estructura de carpetas propuesta

```

app/
  (public)/                                route group — landing actual, sin auth
    page.tsx
    aviso-de-privacidad/
    login/
      page.tsx
      2fa/page.tsx
    password/
      forgot/page.tsx
      reset/page.tsx
  (admin)/
    layout.tsx                            guard de rol ADMIN + layout con sidebar
    admin/
      page.tsx                            dashboard
      lenders/
        page.tsx                          lista

```

new/page.tsx	crear
[id]/	
page.tsx	detalle
edit/page.tsx	
(lender)/	
layout.tsx	guard de rol LENDER + layout con sidebar
lender/	
page.tsx	dashboard
contracts/	
page.tsx	
new/page.tsx	
[id]/	
page.tsx	resumen (con pestañas internas, no sub-rutas — ver
1.3)	
edit/page.tsx	
borrowers/	
page.tsx	
new/page.tsx	
[id]/	
page.tsx	
edit/page.tsx	
(borrower)/	
layout.tsx	guard de rol BORROWER + layout con sidebar
borrower/	
page.tsx	dashboard
contracts/	
[id]/page.tsx	
[id]/accept/page.tsx	aceptación de términos, pantalla dedicada
payments/	
page.tsx	
new/page.tsx	
payment-methods/	
page.tsx	
(auth-onboarding)/	
onboarding/	
2fa-setup/page.tsx	activación obligatoria (Admin/Lender)
change-password/page.tsx	primer login con contraseña temporal
api/	
session/	
login/route.ts	BFF — ver F-2
login-2fa/route.ts	
refresh/route.ts	
logout/route.ts	
logout-all/route.ts	
layout.tsx	root layout (fuentes, tema, providers)
globals.css	
forbidden/page.tsx	403
not-found.tsx	404 (ya soportado por Next)
components/	
ui/	Button, Input, Select, Table, Modal, Toast, Badge, EmptyState, ErrorState, Skeleton, Pagination, ConfirmDialog, Tabs
layout/	Sidebar, Topbar, Breadcrumbs, RoleGuard
forms/	componentes de campo reutilizables

domain/	componentes específicos de dominio: ContractStatusBadge, AmortizationTable, TransactionTimeline, TwoFactorQrCode
lib/	
api/	
client.ts	fetch wrapper + manejo de access token
auth.ts	
admin.ts	
lenders.ts	
borrowers.ts	
contracts.ts	
payments.ts	
session/	
store.ts	contexto/estado de sesión en memoria
permissions.ts	matriz de permisos + hook usePermissions
format.ts	moneda, fecha, porcentaje
types/	
api.ts	tipos calcados 1:1 de la sección 6 del plan de backend
middleware.ts	protección de rutas por rol (edge)
content/	
copy.ts	existente, se extiende con el copy de la app

1.3 Layouts

Tres layouts autenticados (Admin/Lender/Borrower), cada uno con sidebar de navegación propia (ítems distintos por rol, ver sección 6) + topbar con nombre de usuario, logout y estado de conexión. El detalle de un contrato usa **pestañas dentro de la misma página** (Tabs de **components/ui**), no sub-rutas por pestaña, para no perder contexto de scroll/estado al cambiar entre Resumen/Términos/Calendario/Transacciones/Deudores — decisión de UX, no de routing.

1.4 Routing

Next.js App Router con route groups por rol (**(admin)**, **(lender)**, **(borrower)**) para poder aplicar un **layout.tsx** con guard distinto sin que el segmento aparezca en la URL. **middleware.ts** (Next.js Edge Middleware) es la primera línea de defensa de rutas — detalle completo en sección 6.

1.5 Componentes UI reutilizables

Set mínimo, construido una vez y usado en todos los módulos: **Button** (variantes primary/secondary/destructive/ghost, estado **loading**), **Input/Select/Textarea/DatePicker/CurrencyInput/PercentageInput** (con formato y validación inline), **Table** (con soporte de paginación/orden/estado vacío integrado), **Modal**, **ConfirmDialog** (específico para acciones destructivas, ver sección 9), **Toast/NotificationProvider**, **Badge** (para estados de contrato/transacción, con mapeo de color por estado), **EmptyState**, **ErrorState**, **Skeleton**, **Pagination**, **Tabs**.

1.6 Manejo de formularios

Todos los formularios de negocio (crear/editar prestamista, deudor, contrato, términos, método de pago) usan el mismo patrón: schema de validación con Zod (mismas reglas que el backend duplica del lado

cliente para feedback inmediato — la validación real y autoritativa sigue siendo la del backend, ver sección 6), `react-hook-form` para el manejo de estado de campos, componente `<Form>` wrapper que centraliza el estado `idle/submitting/success/error` y el mapeo de errores de campo devueltos por la API (`ApiErrorBody.error.code`) a mensajes visibles.

1.7 Manejo de errores / loading / empty states

Patrón único en toda la app (detalle en sección 9): cada vista de listado/detalle maneja explícitamente sus 4 estados — `loading` (Skeleton), `error` (ErrorState con reintento), `empty` (EmptyState con CTA cuando aplica), `success` (contenido real). Ninguna vista muestra una tabla vacía sin explicar por qué está vacía.

1.8 Manejo de permisos

`lib/session/permissions.ts` expone `can(action, resource)` y un hook `usePermissions()` — matriz completa en sección 6.1. Se usa tanto para ocultar/deshabilitar UI como, indirectamente, para decidir qué rutas construye la navegación de cada sidebar.

2. Autenticación (pantallas y flujos)

Todos los endpoints referenciados son los definidos en `PAYMYLOAN_BACKEND_IMPLEMENTATION_PLAN.md` sección 6.1.

2.1 Login (dos pasos)

`/login`: formulario de `email+password`. Al enviar, llama al BFF `POST /api/session/login` (que internamente llama a `POST /api/auth/login` del backend). Dos resultados posibles:

- Login directo (solo posible para `BORROWER`, que no tiene 2FA): recibe `accessToken` en la respuesta del BFF (se guarda en memoria), cookie de refresh ya seteada por el BFF, redirige según rol (sección 2.8).
- `requiresTwoFactor: true` + `pendingToken`: redirige a `/login/2fa` con el `pendingToken` en estado de navegación (no en la URL, para no dejarlo en el historial del navegador).

2.2 Verificación 2FA

`/login/2fa`: input de 6 dígitos (o un link "usar código de recuperación" que cambia el input a texto libre). Envía a `POST /api/session/login-2fa`. Error de código incorrecto no reinicia el flujo completo, solo limpia el input y muestra el mensaje; tras 3 errores muestra una nota de que hay un límite de intentos (el bloqueo real lo aplica el backend, el frontend solo informa).

2.3 Recuperación de contraseña

`/password/forgot`: un solo campo `email`, siempre muestra el mismo mensaje de éxito exista o no la cuenta (igual que responde el backend) — evita confirmar/negar la existencia de una cuenta desde la UI.

`/password/reset?token=...`: nuevo password + confirmación, mismas reglas de complejidad que valida el backend, mostradas como checklist en vivo (longitud, mayúscula, número) para feedback inmediato.

2.4 Activación de 2FA (obligatoria para Admin/Lender)

`/onboarding/2fa-setup`: pantalla bloqueante — un usuario `ADMIN/LENDER` con `isTwoFactorEnabled=false` es redirigido aquí desde cualquier otra ruta (el middleware lo fuerza, sección 6.3). Tres pasos en una sola pantalla: (1) QR generado a partir de la `otpauth://` URL de `POST /api/auth/2fa/setup` + fallback de código manual para quien no puede escanear, (2) input de verificación que llama `POST /api/auth/2fa/verify`, (3) pantalla de recovery codes — se muestran **una sola vez**, con un checkbox obligatorio "ya los guardé" antes de poder continuar (no hay botón "más tarde").

2.5 Cambio de contraseña temporal (primer login)

`/onboarding/change-password`: aplica a un `LENDER` creado por un Admin o un `BORROWER` creado por un Lender — ambos nacen con contraseña temporal enviada por correo. Pantalla simple de nueva contraseña + confirmación; al completarla, si el rol es `LENDER`, encadena a `/onboarding/2fa-setup` (2FA es obligatorio, no se puede saltar); si es `BORROWER`, va directo a su dashboard.

2.6 Logout

Botón en el topbar de cada layout autenticado → `POST /api/session/logout` (revoca el refresh token actual) → limpia el access token en memoria → redirige a `/login`. Opción adicional "cerrar sesión en todos los dispositivos" en la configuración de cuenta (Admin/Lender) → `POST /api/session/logout-all`.

2.7 Protección de rutas

Ver desarrollo completo en sección 6 — resumen: `middleware.ts` verifica la cookie de refresh (existencia, no validez completa — eso lo hace el backend) antes de renderizar cualquier ruta de `(admin)/(lender)/(borrower)`; cada `layout.tsx` de esos grupos hace una verificación de rol del lado del servidor (llamando `GET /api/auth/me` en un Server Component) antes de renderizar el contenido.

2.8 Redirección post-login según rol

`ADMIN` → `/admin`, `LENDER` (con 2FA activo) → `/lender`, `LENDER` (sin 2FA) → `/onboarding/2fa-setup`, `BORROWER` → `/borrower`. Centralizado en una única función `resolvePostLoginRoute(session)` en `lib/session/store.ts`, para no duplicar la lógica en cada pantalla que redirige tras login.

2.9 Manejo de sesión expirada

Ver interceptor de 401 en sección 7.3: cualquier llamada a la API que reciba 401 dispara un intento de refresh silencioso; si el refresh también falla (refresh token expirado o revocado), se limpia la sesión y se redirige a `/login?reason=session_expired`, mostrando un mensaje ("Tu sesión expiró, inicia sesión de nuevo") en vez de simplemente devolver al login sin explicación.

3. Admin

3.1 Lista de prestamistas

`/admin/lenders` — `Table` con columnas: nombre de empresa, email de contacto, estado (`Badge` Activo/Suspendido), fecha de alta, # de deudores, # de contratos activos. Búsqueda por `companyName/email` (`GET /api/admin/lenders?search=`), filtro por `status`, paginación server-side. Fila clickeable → detalle. Botón "Nuevo prestamista" visible siempre (rol ya garantizado por el layout).

3.2 Vista individual de prestamista

`/admin/lenders/[id]` — cabecera con nombre, estado, fecha de alta, quién lo creó; sección de datos de contacto/dirección de negocio; resumen numérico (deudores totales, contratos por estado); acciones: Editar, Suspend/Reactivar (con `ConfirmDialog`). No se listan aquí los contratos/deudores del prestamista en detalle — el encargo es explícito en que Admin no administra directamente esos recursos; si se necesita visibilidad completa, queda como extensión opcional (FE-036/037), no en el flujo principal de Admin.

3.3 CRUD de prestamistas

- **Crear** (`/admin/lenders/new`): formulario (detalle de campos en sección 8.1) → `POST /api/admin/lenders`. Al completar, muestra explícitamente que se envió una contraseña temporal por correo al prestamista (la API nunca la devuelve).
- **Editar** (`/admin/lenders/[id]/edit`): mismos campos salvo email/contraseña (esos cambian por otro flujo, no por este formulario) → `PATCH /api/admin/lenders/:id`.
- **Suspend/eliminar**: `ConfirmDialog` con copy explícito de consecuencia ("Los contratos activos de este prestamista no se verán afectados, pero no podrá iniciar sesión"); si el backend responde 409 (tiene contratos activos), se muestra el motivo devuelto por la API, no un error genérico.

4. Prestamista

4.1 Contratos

- **Lista** (`/lender/contracts`): `Table` con columnas número de contrato, dirección (resumen: ciudad+estado), estado (`ContractStatusBadge`), saldo actual, próximo vencimiento, deudor(es) principal(es). Búsqueda por número/dirección/nombre de deudor, filtro por `status`, paginación.
- **Crear** (`/lender/contracts/new`): formulario multi-sección en una sola página con navegación por anclas internas (no wizard de varios pasos con "Siguiete" — se prefiere ver todo el formulario porque son ~20 campos relacionados entre sí, no un flujo lineal): Propiedad (dirección), Términos financieros (estructura/monto/tasa/plazo/fechas/mora), Deudores asociados (selector múltiple de deudores existentes del prestamista + opción "crear deudor nuevo" inline, que abre el mismo formulario de FE-050 en un modal). Al enviar, crea `Contract DRAFT` + `ContractTerms v1 DRAFT` (`POST /api/contracts`).
- **Editar** (`/lender/contracts/[id]/edit`): mismos campos; si la versión vigente ya no está `DRAFT`, los campos financieros se deshabilitan con una nota ("Para cambiar estos datos, propone una nueva versión de términos") y solo dirección/metadatos quedan editables — refleja la regla de negocio 8.2 del plan de backend.
- **Eliminar/cancelar**: `DELETE` (solo `DRAFT` sin actividad) o `POST /cancel` (con motivo obligatorio) según el estado — el botón visible cambia de "Eliminar borrador" a "Cancelar contrato" automáticamente según `contract.status`, nunca se muestran ambos.
- **Estados**: reflejados siempre con el mismo `ContractStatusBadge` en toda la app (lista, detalle, dashboard) — un solo componente, un solo mapeo de color, para que el estado de un contrato se lea igual en cualquier pantalla.

4.2 Vista individual de contrato

`/lender/contracts/[id]` — cabecera fija (número, dirección, estado, saldo, próximo pago) + **Tabs**:

1. **Resumen**: todos los datos del contrato y de la versión de términos vigente en modo lectura.
2. **Términos**: historial completo de versiones (`GET /api/contracts/:id/terms`) con quién propuso cada una y el estado de aceptación de cada deudor por versión; botón "Proponer nueva versión" (solo si la vigente no está `DRAFT/PENDING_ACCEPTANCE`).
3. **Calendario**: tabla de amortización completa (`GET /api/contracts/:id/schedule`), con columna de estado por fila y totales.
4. **Transacciones**: historial de pagos reales (`GET /api/contracts/:id/transactions`), con acceso a "Registrar pago manual" para el prestamista.
5. **Deudores**: lista de `ContractBorrower` asociados, con acción de asociar/desasociar.

4.3 Deudores

- **Lista** (`/lender/borrowers`): tabla con nombre, email, teléfono, # de contratos asociados, fecha de alta. Búsqueda + paginación.
- **Crear/Editar/Eliminar**: mismo patrón que prestamistas (sección 3.3) pero tenant-scoped automáticamente — el formulario nunca pide ni muestra un selector de `LenderId` (se resuelve del prestamista autenticado, exactamente como lo garantiza el backend en `POST /api/lenders/me/borrowers`).

4.4 Dashboard del prestamista

`/lender` — resumen de cartera: # de contratos por estado, monto total prestado activo, próximos vencimientos (7/30 días), contratos con mora. Todo con datos reales de la API, ningún número inventado en el diseño — si un endpoint agregado no existe todavía en el backlog de backend, esta vista se construye al final (ver roadmap, sección 11) o con datos calculados client-side sobre la lista ya cargada como fallback documentado.

5. Deudor

5.1 Dashboard

`/borrower` — resumen de su(s) contrato(s): saldo total adeudado, próximo pago (monto+fecha), estado (al día/mora), acceso directo a "Registrar pago". Si tiene un contrato `PENDING_ACCEPTANCE` esperando su decisión, se muestra como banner destacado en la parte superior (no se puede ignorar silenciosamente — ver 5.3).

5.2 Vista de contrato(s) propios

`/borrower/contracts/[id]` — versión de solo lectura del detalle de contrato (mismas pestañas que la vista del prestamista salvo "Deudores" y sin ninguna acción de edición): Resumen, Calendario, Transacciones. Un deudor con más de un contrato ve un selector simple antes de entrar al detalle (no hace falta una lista con tabla/filtros para un volumen que normalmente es de 1–2 contratos por persona).

5.3 Aceptación de términos (pantalla dedicada, bloqueante)

`/borrower/contracts/[id]/accept` — la vista más importante del rol Deudor por la decisión D0-3 del plan de backend. Muestra el detalle completo de la `ContractTerms PENDING_ACCEPTANCE` (monto, tasa,

estructura, calendario resumido, mora) de forma legible (no un JSON, una tabla clara con cada término explicado en una línea), con dos acciones igual de visibles: **Aceptar** y **Rechazar** (este último abre un campo de comentario obligatorio). Ambas acciones muestran un **ConfirmDialog** antes de enviar, porque es una decisión legal/financiera, no una acción reversible con un simple "deshacer". Tras aceptar, si es el último deudor en aceptar, se muestra explícitamente "Tu contrato ya está activo" (el backend genera el calendario automáticamente en ese momento).

5.4 Calendario y pagos

- **Calendario propio** (`/borrower/contracts/[id]` pestaña Calendario): igual que la vista del prestamista, solo lectura.
- **Lista de pagos** (`/borrower/payments`): todas las transacciones propias across todos sus contratos, con estado (**Badge**: en tránsito/exitoso/fallido/devuelto).
- **Registrar pago** (`/borrower/payments/new`): selector de contrato (si tiene más de uno con saldo pendiente), monto (prellenado con el próximo **totalDue**, editable dentro de reglas razonables — el backend valida el monto real), selector de método de pago guardado o "agregar nuevo método" inline. Envía a `POST /api/contracts/:id/payments`.
- **Confirmación de pago**: pantalla de resultado inmediato tras el submit — nunca dice "pago exitoso" de forma optimista; refleja el estado real devuelto (**PENDING/PROCESSING** → mensaje "tu pago está en camino, puede tardar unos días hábiles en confirmarse", nunca "pagado" hasta que el estado real sea **SUCCEEDED**). Este matiz es intencional y viene directamente del defecto que el propio alcance señaló en Owner (sección 4.4 del alcance) — el frontend no debe repetirlo mostrando un pago como confirmado antes de tiempo.

5.5 Método de pago

`/borrower/payment-methods` — lista de métodos guardados (banco, últimos 4 dígitos, estado de verificación), agregar nuevo (flujo de Stripe **SetupIntent**, embebido vía el SDK de Stripe del lado cliente — no se construye un formulario de datos bancarios propio, nunca pasan por los servidores de PayMyLoan), quitar (con confirmación).

5.6 Manejo de errores específico del rol Deudor

Un Deudor sin ningún contrato asociado todavía (cuenta recién creada por un prestamista, sin contrato aún sometido a aceptación) ve un **EmptyState** explícito ("Tu prestamista todavía no te ha asociado a un contrato") en vez de un dashboard vacío sin explicación.

6. Control de acceso

6.1 Matriz de rutas y acciones por rol

Ruta / acción	ADMIN	LENDER	BORROWER
<code>/admin/*</code>	✓	✗	✗
<code>/lender/*</code>	✗	✓ (propio)	✗
<code>/borrower/*</code>	✗	✗	✓ (propio)

Ruta / acción	ADMIN	LENDER	BORROWER
Crear/editar/eliminar Prestamista	✓	✗	✗
Crear/editar/eliminar Deudor	✗	✓ (propios)	✗
Crear/editar/eliminar Contrato	✗	✓ (propios)	✗
Aceptar/rechazar términos	✗	✗	✓ (donde es <code>ContractBorrower</code>)
Registrar pago	✗	✗ (solo manual, ver siguiente fila)	✓ (propio)
Registrar pago manual (wire/cheque)	✗	✓ (sus contratos)	✗
Ver <code>AuditLog</code>	✓ (todo)	✓ (acotado a su tenant, si se construye FE-036)	✗

6.2 Componentes ocultos por permiso

`usePermissions()` decide, además del acceso a rutas completas, la visibilidad de acciones puntuales dentro de una misma vista compartida en el futuro (hoy cada rol tiene layout/rutas separadas, así que el caso más común de "ocultar un botón" es dentro de la vista de detalle de contrato: el botón "Registrar pago manual" solo existe en la variante de la vista que renderiza el layout (`lender`), nunca se renderiza condicionalmente dentro de un mismo componente compartido con lógica de `if (role === ...)` — se prefiere **componer** vistas por rol reutilizando sub-componentes de solo-lectura, no ramificar un único componente con muchos `if`.

6.3 Acceso manual a una URL no permitida

Tres capas, de más temprana a más tardía:

1. **middleware.ts** (edge, antes de renderizar nada): si no hay cookie de sesión en absoluto y la ruta requiere auth → redirect a `/login`. No puede validar el rol aquí de forma barata (no decodifica el JWT completo en el edge para no duplicar la lógica de verificación del backend) — solo bloquea "sin sesión".
2. **layout.tsx de cada route group** (Server Component): llama `GET /api/auth/me` con la sesión; si `role` no coincide con el grupo (ej. un `BORROWER` cargando `/lender/contracts`), redirige a `/forbidden` con el rol real detectado.
3. **Backend**: la autoridad final — si por cualquier razón una request de API se ejecuta igual, el backend responde 403/404 (sección 7.5 del plan de backend) y el frontend lo traduce al mismo `ErrorState`/página 403 (sección 6.4).

6.4 Manejo de 401/403 del backend

- **401** en cualquier llamada a la API (fuera del propio flujo de login): interceptor intenta un refresh silencioso una vez; si falla, limpia sesión y redirige a `/login?reason=session_expired` (sección 2.9).

- **403** (rol correcto pero acción no permitida, ej. 2FA pendiente): redirige a `/onboarding/2fa-setup` si el código es `TWO_FACTOR_REQUIRED`; para cualquier otro 403, página `/forbidden` genérica con el mensaje que devuelva la API.
- **404 por mismatch de tenant** (un Lender/Borrower probando un ID ajeno): se trata igual que un 404 real — "Este recurso no existe o no tienes acceso" — nunca se distingue en la UI entre "no existe" y "no es tuyo", exactamente por la misma razón de anti-enumeración que ya aplica el backend (sección 7.5 del plan de backend).

Nota explícita (pedida por el encargo): nada de lo anterior es la capa de seguridad real — es UX. El frontend oculta botones y bloquea rutas para que la experiencia sea coherente, pero la autorización que importa (y la que se prueba con la suite de tests multi-tenant, sección 11 del plan de backend) es la del backend. Un usuario que edite el JavaScript en su navegador o llame a la API directo con `curl` sigue sujeto exactamente a las mismas reglas — el frontend nunca es, ni debe tratarse como, la única barrera.

7. Integración con backend

7.1 Cliente HTTP

`lib/api/client.ts`: wrapper sobre `fetch` con `baseUrl=process.env.NEXT_PUBLIC_API_URL`, adjunta `Authorization: Bearer <accessToken>` (leído del store en memoria, sección 7.2), serializa/deserializa JSON, normaliza toda respuesta al tipo `ApiResponse<T> | ApiErrorBody` que ya define `paymyloan-back` (mismo contrato que documenta su propio README) — nunca se asume una forma de respuesta distinta por endpoint.

7.2 Manejo de tokens/sesión

- **Access token**: variable en memoria dentro de un React Context (`lib/session/store.ts`), nunca en `localStorage/sessionStorage` — se pierde al refrescar la página por diseño (mitiga XSS) y se recupera automáticamente vía `POST /api/session/refresh` al montar la app (lee la cookie `httpOnly` del lado servidor).
- **Refresh token**: cookie `httpOnly; Secure; SameSite=Strict`, seteada exclusivamente por los Route Handlers de `/api/session/*` (nunca accesible desde JS del cliente).
- **BFF de sesión** (`app/api/session/*/route.ts`): los únicos 4 endpoints del frontend que hablan con el backend desde el servidor de Next.js en vez de desde el navegador — `login`, `login-2fa`, `refresh`, `logout/logout-all`. Todo lo demás (contratos, deudores, pagos, etc.) es llamada directa navegador → `paymyloan-back` con el access token en memoria.

7.3 Manejo de errores / interceptors

`client.ts` intercepta toda respuesta 401: dispara `POST /api/session/refresh` una vez, si obtiene un access token nuevo reintenta la request original una sola vez, si el refresh también falla limpia la sesión y redirige (sección 2.9). Errores 4xx/5xx no relacionados con auth se propagan como una excepción tipada (`ApiError { code, message, status }`) que cada pantalla decide cómo mostrar (mensaje inline en formularios, `ErrorState` en vistas de listado/detalle).

7.4 Tipos/interfaces compartidos

`lib/types/api.ts` — se mantienen manualmente en sincronía con la sección 6 del plan de backend (no hay generación automática de tipos desde el backend en este plan, porque `paymyloan-back` no expone un contrato OpenAPI/tRPC hoy — se documenta como mejora futura opcional en la sección 12, no como bloqueo). Cada tipo de dominio (`Contract`, `ContractTerms`, `Transaction`, etc.) se nombra igual que el modelo Prisma correspondiente para que buscar "Contract" en ambos repos encuentre lo mismo conceptualmente.

7.5 Estrategia de sincronización/refetch

Sin librería de data-fetching con cache (React Query/SWR) en el alcance mínimo de este plan — se usan Server Components para la carga inicial de cada vista (menos JS en el cliente, más simple) y `router.refresh()`/refetch manual tras mutaciones (crear/editar/eliminar) para reflejar el estado nuevo, en vez de actualizar cache local optimista. Se documenta como decisión deliberada de simplicidad para el MVP — si el volumen de interacciones lo justifica después, migrar a React Query es un cambio localizado en `lib/api/*`, no una reescritura de las vistas.

8. Formularios

Detalle de los formularios más importantes — el resto (editar prestamista, editar deudor) reutiliza el mismo patrón con un subconjunto de campos.

8.1 Crear/editar Prestamista

Campo	Tipo	Obligatorio	Validación
companyName	texto	Sí	2–120 caracteres
email	email	Sí (solo en crear)	formato válido, verificado único por la API (409 → mensaje inline en el campo)
contactPhone	texto	No	formato de teléfono US
addressLine1 / city / state / postalCode	texto	No	—
notes	textarea	No	máx 2000 caracteres

Estados: `idle` → `submitting` → `success` (redirige al detalle) / `error` (mensaje inline + toast).

8.2 Crear/editar Deudor

Campo	Tipo	Obligatorio	Validación
name	texto	Sí	2–120 caracteres
email	email	Sí (solo en crear)	único, verificado por la API
phone	texto	No	
addressLine1 / city / state / postalCode	texto	No	
notes	textarea	No	

8.3 Crear/editar Contrato

Campo	Tipo	Obligatorio	Validación
addressLine1	texto	Sí	
addressLine2	texto	No	
city / state / postalCode	texto	Sí	state selector de 2 letras
county	texto	No	
propertyType	select	Sí	enum PropertyType
parcelNumber	texto	No	
structure	select	Sí	INTEREST_ONLY/AMORTIZED/BALLOON — cambia qué otros campos se muestran/requieren
principalAmount	moneda	Sí	> 0
interestRate	porcentaje	Sí	> 0, ≤ 2 decimales de precisión mostrados
dayCountConvention	select	Sí	default THIRTY_360
amortizationTermMonths	número	Sí	> 0, visible siempre pero especialmente relevante en BALLOON
firstPaymentDate	fecha	Sí	≥ hoy
paymentDueDay	número	Sí	1–31
maturityDate	fecha	Sí	> firstPaymentDate
lateFeeType	select	Sí	FLAT/PERCENTAGE
lateFeeAmount	moneda o porcentaje según lateFeeType	Sí	> 0
gracePeriodDays	número	Sí	default 10, ≥ 0
deudores	multi-select	Sí, al menos 1	de la lista de deudores propios del prestamista, con opción de crear uno nuevo inline

Estados: **idle** → **submitting** → **success** (redirige al detalle, muestra banner "términos en borrador, recuerda enviarlos a aceptación") / **error**. La validación cliente (Zod) espeja las reglas del backend descritas en el plan de backend (BE-051) para dar feedback antes del submit, pero el submit siempre revalida contra la respuesta real de la API.

8.4 Registrar pago (Deudor)

Campo	Tipo	Obligatorio	Validación
contractId	select (si aplica)	Sí	solo contratos ACTIVE / DELINQUENT propios
amount	moneda	Sí	> 0, prellenado con totalDue de la próxima fila pendiente
paymentMethodId	select	Sí	de los métodos guardados verificados; si no hay ninguno, redirige primero a agregar uno

Estados: **idle** → **submitting** → **processing** (con mensaje explícito de que puede tardar) → **success/error**, nunca colapsa **processing** y **success** en un solo estado visual (sección 5.4).

8.5 Aceptar/rechazar términos (Deudor)

Sin campos de "aceptar" (es un botón de acción, no un formulario); "rechazar" tiene un único campo obligatorio: **comment** (textarea, máx 1000 caracteres, "cuéntale al prestamista qué te gustaría cambiar").

9. UX

9.1 Loading states

Skeleton con la forma aproximada del contenido real (filas de tabla, tarjetas de resumen) en toda carga inicial — nunca un spinner genérico centrado en pantalla completa para contenido que ya tiene un layout conocido.

9.2 Empty states

Cada listado (**Table**) tiene un **EmptyState** propio y específico del contexto (no un genérico "No hay datos"): lista de prestamistas vacía → "Crea el primer prestamista para empezar" + CTA; lista de contratos de un prestamista nuevo → "Todavía no tienes contratos" + CTA; deudor sin contratos → mensaje explicado en 5.6.

9.3 Error states

ErrorMessage con: mensaje humano (nunca el **message** crudo de un error 500), código de referencia visible solo si es útil para soporte, botón de reintento que vuelve a disparar el fetch original. Errores de validación de formulario nunca se muestran como **ErrorMessage** de página completa — siempre inline en el campo correspondiente o como resumen arriba del formulario.

9.4 Confirmaciones para acciones destructivas

ConfirmDialog obligatorio para: eliminar/cancelar contrato, eliminar deudor, suspender prestamista, desasociar deudor de contrato, quitar método de pago, rechazar términos, cerrar sesión en todos los dispositivos. Cada uno con copy específico de la consecuencia real (no un genérico "¿Estás seguro?") — ej. cancelar un contrato **ACTIVE** explica que el calendario y transacciones existentes se conservan pero no se generan más cobros.

9.5 Feedback después de operaciones

Toda mutación exitosa dispara un **Toast** de confirmación específico ("Prestamista creado", "Términos enviados a aceptación") además de la navegación/actualización de la vista — nunca se navega en silencio sin confirmar que la acción ocurrió.

9.6 Responsive design

Mobile-first para las vistas del Deudor (es plausible que revise su deuda y pague desde el teléfono); las vistas de Admin/Prestamista se optimizan primero para escritorio (tablas densas, formularios largos) con un breakpoint razonable para tablet, sin comprometer la usabilidad de las tablas en mobile (scroll horizontal contenido dentro de la tabla, nunca desbordando la página — mismo criterio que ya sigue la landing actual con overflow controlado).

9.7 Accesibilidad básica

Contraste verificado contra los tokens de color ya definidos en **globals.css** (que ya declaran variantes clara/oscura), navegación por teclado en todos los componentes interactivos (**Modal/ConfirmDialog** con foco atrapado y cierre con **Esc**), etiquetas **aria-label** en botones de solo ícono (ej. acciones de tabla), **:focus-visible** ya definido globalmente en el CSS existente — se reutiliza y se aplica a los componentes nuevos.

10. Backlog detallado

Convención de cada ítem: **Objetivo**, **Dependencias**, **Componentes**, **Rutas**, **Integración API**, **Estados UI**, **Validaciones**, **Criterios de aceptación**.

Fase 1 — Foundation

FE-001 — Cliente HTTP base

- **Objetivo:** **lib/api/client.ts** con manejo de **Authorization**, parseo de **ApiSuccessBody/ApiErrorBody**, y el interceptor de 401 (sin el refresh todavía, eso es FE-008).
- **Dependencias:** ninguna (puede ir en paralelo al backend, contra el contrato ya documentado).
- **Componentes:** N/A (librería).
- **Integración API:** **NEXT_PUBLIC_API_URL**.
- **Criterios de aceptación:** una llamada a un endpoint inexistente del backend real (o un mock) produce un **ApiError** tipado, no una excepción sin capturar.

FE-002 — BFF de sesión

- **Objetivo:** Route Handlers **app/api/session/{login,login-2fa,refresh,logout,logout-all}/route.ts**.
- **Dependencias:** backend BE-027 a BE-030 disponibles (o mockeados).
- **Integración API:** proxy directo a **/api/auth/*** del backend, seteo/lectura de cookie httpOnly.
- **Criterios de aceptación:** la cookie de refresh nunca es legible desde **document.cookie** en el navegador (verificado manualmente en devtools).

FE-003 — Tipos compartidos

- **Objetivo:** `Lib/types/api.ts` calcado de la sección 6 del plan de backend.
- **Dependencias:** ninguna.
- **Criterios de aceptación:** cero uso de `any` en las funciones de `Lib/api/*` una vez creadas.

FE-004 — Extender sistema de diseño

- **Objetivo:** nuevos tokens/utilidades necesarios para UI densa de app (tabla, sidebar) sin romper los ya existentes de la landing.
- **Dependencias:** ninguna.
- **Componentes:** `app/globals.css`.
- **Criterios de aceptación:** la landing pública sigue viéndose idéntica tras el cambio (regresión visual manual).

FE-005 — Librería de componentes UI base

- **Objetivo:** `components/ui/*` — lista completa en sección 1.5.
- **Dependencias:** FE-004.
- **Criterios de aceptación:** cada componente tiene al menos un estado de error/loading/vacío donde aplique, documentado con un ejemplo de uso mínimo (no se exige Storybook, pero sí un caso de uso de referencia en el propio código).

FE-006 — Estructura de carpetas app router por rol

- **Objetivo:** crear los route groups vacíos (`(admin)`, `(lender)`, `(borrower)`, `(public)`) con layouts placeholder.
- **Dependencias:** ninguna.
- **Criterios de aceptación:** `pnpm build` pasa con las rutas nuevas vacías.

FE-007 — Store de sesión

- **Objetivo:** `Lib/session/store.ts` — Context de React con `{ user, role, accessToken, isLoading }` + `login()/logout()/refresh()`.
- **Dependencias:** FE-002.
- **Criterios de aceptación:** recargar la página recupera la sesión automáticamente vía refresh silencioso si la cookie sigue válida.

FE-008 — Interceptor de 401 + refresh + reintento

- **Objetivo:** completar `client.ts` con la lógica descrita en sección 7.3.
- **Dependencias:** FE-001, FE-002, FE-007.
- **Criterios de aceptación:** una request que recibe 401 se reintenta automáticamente una vez tras un refresh exitoso, sin que la pantalla que la disparó tenga que saberlo.

FE-009 — Sistema de notificaciones/toasts

- **Objetivo:** `NotificationProvider` + hook `useToast()`.
- **Dependencias:** FE-005.

FE-010 — Variables de entorno

- **Objetivo:** `.env.example` del frontend con `NEXT_PUBLIC_API_URL` y cualquier clave pública de Stripe necesaria para el SDK cliente.
- **Dependencias:** ninguna.

FE-011 — Utilidades de formato

- **Objetivo:** `lib/format.ts` — moneda (USD), fecha, porcentaje, consistentes con los `Decimal` del backend (nunca se hace aritmética de dinero en el frontend, solo formato de presentación).
- **Dependencias:** ninguna.

FE-012 — Hook de permisos

- **Objetivo:** `lib/session/permissions.ts` — matriz de sección 6.1 como estructura de datos + `usePermissions()`.
- **Dependencias:** FE-007.
- **Criterios de aceptación:** `can('contract:delete', contract)` devuelve `false` para un rol `BORROWER` sin necesidad de consultar la API.

Fase 2 — Authentication

FE-013 — Página de login (paso 1)

- **Objetivo:** `/login` — formulario de credenciales.
- **Dependencias:** FE-001, FE-002, FE-005.
- **Rutas:** `(public)/login/page.tsx`.
- **Integración API:** `POST /api/session/login`.
- **Estados UI:** `idle/submitting/error` (credenciales inválidas)/`redirect` (éxito o 2FA requerido).
- **Validaciones:** email formato válido, password no vacío.
- **Criterios de aceptación:** credenciales inválidas muestran el mismo mensaje genérico sin distinguir "email no existe" de "password incorrecta".

FE-014 — Página de verificación 2FA

- **Objetivo:** `/login/2fa`.
- **Dependencias:** FE-013.
- **Integración API:** `POST /api/session/login-2fa`.
- **Estados UI:** incluye modo "usar código de recuperación".
- **Criterios de aceptación:** recargar la página sin `pendingToken` en el estado de navegación redirige de vuelta a `/login` (no se puede acceder directo por URL).

FE-015 — Recuperación de contraseña (solicitar)

- **Objetivo:** `/password/forgot`.
- **Dependencias:** FE-001.
- **Integración API:** `POST /api/auth/password/forgot`.
- **Criterios de aceptación:** mensaje de éxito idéntico exista o no la cuenta.

FE-016 — Recuperación de contraseña (reset)

- **Objetivo:** `/password/reset?token=`.
- **Dependencias:** FE-015.
- **Integración API:** `POST /api/auth/password/reset`.
- **Validaciones:** checklist en vivo de complejidad.
- **Criterios de aceptación:** token inválido/expirado muestra un mensaje claro con link para pedir uno nuevo, no un error genérico.

FE-017 — Activación de 2FA obligatoria

- **Objetivo:** `/onboarding/2fa-setup`.
- **Dependencias:** FE-005, FE-007.
- **Integración API:** `POST /api/auth/2fa/setup`, `POST /api/auth/2fa/verify`.
- **Componentes:** `TwoFactorQrCode`.
- **Criterios de aceptación:** no se puede salir de la pantalla (ni navegar a otra ruta) sin completar la verificación, salvo logout.

FE-018 — Cambio de contraseña temporal

- **Objetivo:** `/onboarding/change-password`.
- **Dependencias:** FE-007.
- **Integración API:** endpoint de cambio de contraseña propio (`PATCH` sobre el usuario autenticado).
- **Criterios de aceptación:** tras completar, `LENDER` encadena a FE-017, `BORROWER` va a `/borrower`.

FE-019 — Logout / logout-all

- **Objetivo:** acción en topbar + opción en configuración de cuenta.
- **Dependencias:** FE-002, FE-007.

FE-020 — Middleware de protección de rutas

- **Objetivo:** `middleware.ts` — capa 1 de sección 6.3.
- **Dependencias:** FE-002.
- **Criterios de aceptación:** acceder a `/admin` sin cookie de sesión redirige a `/login` antes de que se ejecute cualquier Server Component.

FE-021 — Manejo de sesión expirada

- **Objetivo:** `/login?reason=session_expired` con mensaje contextual.
- **Dependencias:** FE-008.

FE-022 — Redirección post-login por rol

- **Objetivo:** `resolvePostLoginRoute(session)`.
- **Dependencias:** FE-007.

FE-023 — Guard de 2FA pendiente

- **Objetivo:** cualquier ruta (`admin`)/(`lender`) distinta de `/onboarding/*` redirige a `/onboarding/2fa-setup` si `isTwoFactorEnabled=false`.
- **Dependencias:** FE-017, layouts de sección 3.

FE-024 — Pantalla de recovery codes

- **Objetivo:** parte de FE-017, se lista aparte porque tiene su propio criterio de aceptación.
- **Criterios de aceptación:** los códigos solo se muestran una vez en toda la vida de la sesión — recargar la página no los vuelve a mostrar (se pierden del estado en memoria a propósito).

Fase 3 — Layouts

FE-025 — Layout Admin

- **Objetivo:** sidebar (Prestamistas, [futuro: Auditoría]), topbar, guard de rol.
- **Dependencias:** FE-012, FE-006.

FE-026 — Layout Prestamista

- **Objetivo:** sidebar (Dashboard, Contratos, Deudores), topbar, guard de rol + guard de 2FA (FE-023).
- **Dependencias:** FE-012, FE-006.

FE-027 — Layout Deudor

- **Objetivo:** sidebar (Dashboard, Mis contratos, Pagos, Método de pago), topbar, guard de rol.
- **Dependencias:** FE-012, FE-006.

FE-028 — Breadcrumbs/header contextual

- **Objetivo:** componente compartido por los 3 layouts.
- **Dependencias:** FE-025–027.

FE-029 — Layout público

- **Objetivo:** reutilizar `SiteShell/SiteHeader` existentes para las pantallas de auth, sin sidebar de app.
- **Dependencias:** ninguna (componentes ya existen).

Fase 4 — Admin

FE-030 — Lista de prestamistas

- **Objetivo:** sección 3.1.
- **Dependencias:** FE-025, FE-005.
- **Integración API:** `GET /api/admin/lenders`.
- **Estados UI:** loading/error/empty/success.
- **Criterios de aceptación:** búsqueda y paginación funcionan combinadas (buscar dentro de una página distinta a la 1 resetea a la página 1).

FE-031 — Vista individual de prestamista

- **Objetivo:** sección 3.2.
- **Dependencias:** FE-030.
- **Integración API:** `GET /api/admin/lenders/:id`.

FE-032 — Formulario crear prestamista

- **Objetivo:** sección 8.1.
- **Dependencias:** FE-030.
- **Integración API:** `POST /api/admin/lenders`.
- **Validaciones:** sección 8.1.
- **Criterios de aceptación:** email duplicado (409 del backend) se muestra como error inline en el campo `email`, no como toast genérico.

FE-033 — Formulario editar prestamista

- **Objetivo:** igual a FE-032 sin email/password.
- **Dependencias:** FE-031.
- **Integración API:** `PATCH /api/admin/lenders/:id`.

FE-034 — Confirmación desactivar prestamista

- **Objetivo:** `ConfirmDialog` + manejo del 409 si tiene contratos activos (sección 3.3).
- **Dependencias:** FE-031.
- **Integración API:** `DELETE /api/admin/lenders/:id`.

FE-035 — Dashboard admin

- **Objetivo:** métricas básicas (# prestamistas activos, altas recientes).
- **Dependencias:** FE-030.

FE-036 — (Opcional) Vista global de auditoría

- **Objetivo:** `GET /api/audit-logs` sin filtro de tenant.
- **Dependencias:** backend BE-072.
- **Nota:** no forma parte del roadmap mínimo — ver sección 12.

FE-037 — (Opcional) Vista global de contratos de solo lectura

- **Objetivo:** visibilidad de Admin sobre todos los contratos si se confirma que la hace falta (el encargo dice explícitamente que Admin no administra contratos — esta vista, de existir, sería estrictamente de lectura/soporte).
- **Nota:** no forma parte del roadmap mínimo, sujeta a confirmación.

FE-038 — Empty state de prestamistas

- **Objetivo:** parte de FE-030, listado aparte por claridad de backlog.

Fase 5 — Prestamista (Contratos + Deudores)

FE-039 — Lista de contratos

- **Objetivo:** sección 4.1.
- **Dependencias:** FE-026, FE-012.
- **Integración API:** `GET /api/contracts`.
- **Criterios de aceptación:** filtro por estado usa exactamente los valores de `ContractStatus` del backend, sin duplicar el enum a mano de forma que puedan desincronizarse (se importa desde `Lib/types/api.ts`).

FE-040 — Formulario crear contrato

- **Objetivo:** sección 4.1 y 8.3.
- **Dependencias:** FE-039, FE-049 (necesita poder crear un deudor inline).
- **Integración API:** `POST /api/contracts`, `POST /api/lenders/me/borrowers` (para el flujo inline).
- **Validaciones:** sección 8.3 completa.
- **Criterios de aceptación:** `maturityDate` anterior a `firstPaymentDate` se bloquea en el cliente antes de llegar al backend.

FE-041 — Formulario editar contrato

- **Objetivo:** sección 4.1.
- **Dependencias:** FE-040.
- **Integración API:** `PATCH /api/contracts/:id`.
- **Criterios de aceptación:** campos financieros deshabilitados cuando `currentTerms.status !== 'DRAFT'`, con la nota explicativa visible (no solo `disabled` sin contexto).

FE-042 — Confirmación eliminar/cancelar contrato

- **Objetivo:** sección 4.1.
- **Dependencias:** FE-039.
- **Integración API:** `DELETE /api/contracts/:id` o `POST /api/contracts/:id/cancel` según estado.

FE-043 — Vista individual de contrato — Resumen

- **Objetivo:** sección 4.2, pestaña 1.
- **Dependencias:** FE-039.
- **Integración API:** `GET /api/contracts/:id`.

FE-044 — Vista individual — Términos

- **Objetivo:** sección 4.2, pestaña 2.
- **Dependencias:** FE-043.
- **Integración API:** `GET /api/contracts/:id/terms`.

FE-045 — Vista individual — Calendario

- **Objetivo:** sección 4.2, pestaña 3.
- **Dependencias:** FE-043.

- Integración API: `GET /api/contracts/:id/schedule`.
- Componentes: `AmortizationTable`.

FE-046 — Vista individual — Transacciones

- Objetivo: sección 4.2, pestaña 4.
- Dependencias: FE-043.
- Integración API: `GET /api/contracts/:id/transactions`.
- Componentes: `TransactionTimeline`.
- Criterios de aceptación: incluye la acción "Registrar pago manual" solo visible para `LENDER`.

FE-047 — Vista individual — Deudores

- Objetivo: sección 4.2, pestaña 5.
- Dependencias: FE-043.
- Integración API: `POST/DELETE /api/contracts/:id/borrowers`.

FE-048 — Flujo "proponer nueva versión de términos"

- Objetivo: reutiliza el formulario de FE-040/041 para los campos financieros, con `changeSummary` obligatorio.
- Dependencias: FE-044.
- Integración API: `POST /api/contracts/:id/terms` + submit.
- Criterios de aceptación: al enviar, la pestaña Términos refleja de inmediato la nueva versión en `PENDING_ACCEPTANCE` y notifica en el resumen que el contrato espera aceptación del/los deudor(es).

FE-049 — Lista de deudores

- Objetivo: sección 4.3.
- Dependencias: FE-026.
- Integración API: `GET /api/lenders/me/borrowers`.

FE-050 — Formulario crear deudor

- Objetivo: sección 8.2.
- Dependencias: FE-049.
- Integración API: `POST /api/lenders/me/borrowers`.

FE-051 — Formulario editar deudor

- Objetivo: sección 8.2.
- Dependencias: FE-050.
- Integración API: `PATCH /api/lenders/me/borrowers/:id`.

FE-052 — Confirmación eliminar deudor

- Objetivo: sección 4.3.
- Dependencias: FE-049.

- **Integración API:** **DELETE** `/api/lenders/me/borrowers/:id`.
- **Criterios de aceptación:** 409 del backend (contratos activos) se muestra con el motivo, no como error genérico.

FE-053 — Asociar/desasociar deudor a contrato

- **Objetivo:** parte de FE-047, listado aparte por su propio criterio de aceptación.
- **Criterios de aceptación:** el selector de deudores a asociar solo muestra deudores del mismo prestamista (garantizado por el backend, pero el frontend tampoco debe listar deudores ajenos aunque sea información que nunca llegaría del API).

FE-054 — Dashboard prestamista

- **Objetivo:** sección 4.4.
- **Dependencias:** FE-039, FE-049.

Fase 6 — Deudor

FE-055 — Dashboard deudor

- **Objetivo:** sección 5.1.
- **Dependencias:** FE-027.
- **Integración API:** **GET** `/api/contracts` (rol BORROWER).

FE-056 — Vista de contrato(s) propios

- **Objetivo:** sección 5.2.
- **Dependencias:** FE-055.
- **Integración API:** **GET** `/api/contracts/:id` (variante solo lectura de FE-043/045/046).

FE-057 — Pantalla de aceptación/rechazo de términos

- **Objetivo:** sección 5.3 — la más importante del rol.
- **Dependencias:** FE-056.
- **Integración API:** **POST** `/api/contracts/:id/terms/:termsId/accept, /reject`.
- **Estados UI:** incluye el estado "ya decidiste, esperando a los demás deudores" si hay co-deudores.
- **Criterios de aceptación:** rechazar sin **comment** está bloqueado en el cliente (backend también lo validaría, pero no debe depender solo de eso).

FE-058 — Vista de calendario de pagos propio

- **Objetivo:** parte de FE-056.
- **Dependencias:** FE-056.

FE-059 — Lista de transacciones/pagos propios

- **Objetivo:** sección 5.4.
- **Dependencias:** FE-027.

- **Integración API:** `GET /api/contracts/:id/transactions` agregado por contrato, o un endpoint agregado propio si se confirma que hace falta (ver nota en sección 12).

FE-060 — Formulario registrar pago

- **Objetivo:** sección 8.4.
- **Dependencias:** FE-062 (necesita al menos un método de pago).
- **Integración API:** `POST /api/contracts/:id/payments`.

FE-061 — Confirmación de pago

- **Objetivo:** sección 5.4, matiz de estados reales vs optimistas.
- **Dependencias:** FE-060.
- **Criterios de aceptación:** nunca muestra "pagado" cuando el estado real es `PENDING/PROCESSING`.

FE-062 — Gestión de método de pago

- **Objetivo:** sección 5.5.
- **Dependencias:** FE-027, SDK de Stripe del lado cliente.
- **Integración API:** `POST/GET/DELETE /api/payment-methods`.
- **Nota:** bloqueado por la misma decisión de negocio (Stripe Connect, A-3 del plan de backend) que bloquea BE-065/066 — el formulario y la UI se pueden construir contra un mock mientras tanto.

FE-063 — Estado de cuenta simple

- **Objetivo:** sección 5.4/8.4, vista de solo lectura resumida.
- **Dependencias:** FE-056.
- **Nota:** prioridad baja, no bloquea el resto del rol Deudor.

Fase 7 — Permissions

FE-064 — Componente `<RequireRole>/<Can>`

- **Objetivo:** envoltorio declarativo sobre `usePermissions()` para ocultar sub-árboles de UI.
- **Dependencias:** FE-012.

FE-065 — Página 403

- **Objetivo:** `/forbidden`.
- **Dependencias:** FE-012.

FE-066 — Manejo de acceso manual a URL no permitida

- **Objetivo:** capas 1–2 de sección 6.3 integradas end-to-end.
- **Dependencias:** FE-020, FE-025–027.
- **Criterios de aceptación:** un `BORROWER` que edita la URL a mano hacia `/admin/lenders` termina en `/forbidden`, no en un error de render o una pantalla en blanco.

FE-067 — Ocultar/deshabilitar acciones según permisos

- **Objetivo:** aplicar FE-064 a los botones de acción identificados en la matriz de sección 6.1.
- **Dependencias:** FE-064, todas las vistas de fases 3–6.

Fase 8 — Testing

FE-068 — Setup de testing

- **Objetivo:** Vitest + React Testing Library (componentes/unit) + Playwright (E2E).
- **Dependencias:** ninguna, se puede hacer en paralelo a Foundation.

FE-069 — Tests de componentes UI base

- **Objetivo:** `Table`, `Form`, `ConfirmDialog` — estados y accesibilidad básica (foco, `aria-label`).
- **Dependencias:** FE-005, FE-068.

FE-070 — Tests de flujo de autenticación

- **Objetivo:** login, 2FA, refresh silencioso, logout — contra el backend real de desarrollo o un mock de sus respuestas.
- **Dependencias:** FE-013–024, FE-068.

FE-071 — Tests de control de acceso

- **Objetivo:** cada ruta protegida rechaza correctamente a los roles no permitidos (espeja la matriz de sección 6.1).
- **Dependencias:** FE-066, FE-068.

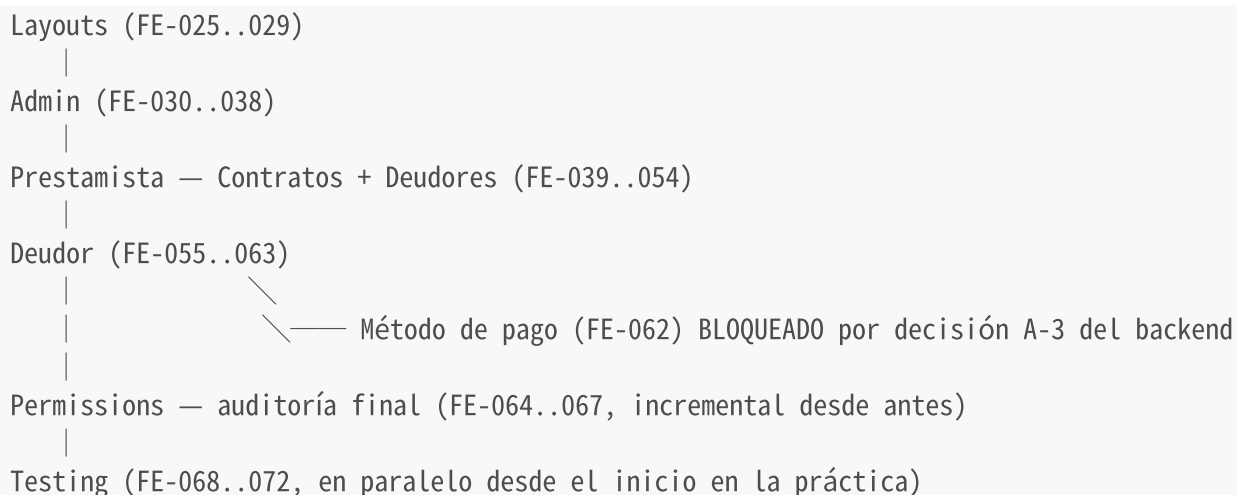
FE-072 — Tests E2E de flujos críticos

- **Objetivo:** Playwright — "Prestamista crea contrato → Deudor lo acepta → contrato queda activo → Deudor registra un pago manual (vía Lender) → aparece en el historial", de punta a punta contra el stack real en Docker.
- **Dependencias:** todas las fases funcionales completas, backend Fase 6/7 disponible.

11. Roadmap recomendado

El orden pedido por el encargo (Foundation → Authentication → Layouts → Admin → Lender → Contracts → Borrowers → Payments → Permissions → Testing) se mantiene casi tal cual, con un ajuste: **Borrowers** (dentro del módulo Prestamista) se agrupa junto con Contracts en la Fase 5 porque en este modelo un contrato no se puede crear sin al menos un deudor asociado — construirlos por separado obligaría a mockear deudores para poder probar contratos. Permissions (Fase 7) se implementa incrementalmente desde Foundation (FE-012) y se completa/audita al final, no se construye de cero recién en esa fase.

```
Foundation (FE-001..012)
|
Authentication (FE-013..024)
|
```



12. Riesgos y decisiones pendientes

#	Pendiente	Impacto si no se resuelve
1	F-1/F-2/F-3 (arquitectura propia de este plan) no fueron confirmadas explícitamente por el encargo	Son de bajo riesgo/reversibles (F-1 y F-2 son decisiones de implementación interna; F-3 solo pospone trabajo de traducción), pero se listan para que puedan objetarse antes de empezar
2	Stripe Connect sin resolver (heredado del plan de backend, A-3)	Bloquea FE-062 y la parte de Stripe de FE-060; el resto del flujo de pagos (registro manual, vista de historial) no depende de esto
3	Bilingüismo (alcance Q13)	Si se confirma que sí hace falta, agrega trabajo de traducción y de UI (toggle) a cada pantalla nueva — no es una re-arquitectura gracias a F-3, pero sí es esfuerzo adicional no presupuestado en este backlog
4	Endpoint agregado de pagos "todas mis transacciones across contratos" (FE-059)	El plan de backend no define explícitamente un endpoint así (solo <code>GET /api/contracts/:id/transactions</code> , por contrato) — si un deudor con varios contratos necesita una vista unificada, hace falta un endpoint nuevo no listado en el backlog de backend; se documenta aquí para que se agregue si se confirma que es necesario
5	Vistas opcionales de Admin (FE-036/037)	El encargo es explícito en que Admin no administra contratos/deudores directamente — estas vistas solo deberían construirse si se confirma que Admin necesita visibilidad de solo lectura para soporte
6	Sin librería de cache de datos (React Query/SWR) (sección 7.5)	Decisión deliberada de simplicidad para el MVP — si la app crece en interactividad (ediciones concurrentes, necesidad de cache real), es una migración futura a planificar aparte

#	Pendiente	Impacto si no se resuelve
7	Destino del formulario de registro actual de la landing (<code>RegisterForm.tsx</code>)	No está definido si debe seguir existiendo tal cual, redirigir a <code>/login</code> , o convertirse en un flujo de alta de Prestamista con aprobación de Admin — el encargo no lo menciona y no se decide unilateralmente aquí

Fin del plan. Ningún archivo del proyecto (`app/`, `content/`, configuración de Docker/nginx) fue modificado como parte de este documento — es exclusivamente material de planificación.